globoplay

# SPRINT 04

vale a pena ver de novo

inteli

# AGENDA

Arquitetura Final da Solução

Conteinerização do projeto

Testes Funcionais do Sistema

Próximos passos



globoplay

# ARQUITETURA FINAL DA SOLUÇÃO


VALE A PENA
VER
DE NOVO

# A ARQUITETURA ANTERIOR

**NAVEGADOR**
o usuário

**PAINEL NO S3**
arquivos do site, servidos direto do armazenamento

**BALANCEADOR**
reparte os acessos entre as máquinas

**DUAS MÁQUINAS VIRTUAIS**
cada máquina subia o serviço sozinha, no momento em que ligava

**BANCO DE DADOS**
onde as informações ficam guardadas

# O PROBLEMA DESSA ARQUITETURA

### SEM UM GERENTE

Cada máquina subia o serviço por conta própria, na mão. Se o serviço travava, ninguém o levantava de volta.

### RECUPERAÇÃO MUITO LENTA

Repor uma máquina que caiu levava até 5 minutos: ligar, instalar, baixar, subir.

### ATUALIZAÇÕES MUITO MANUAIS

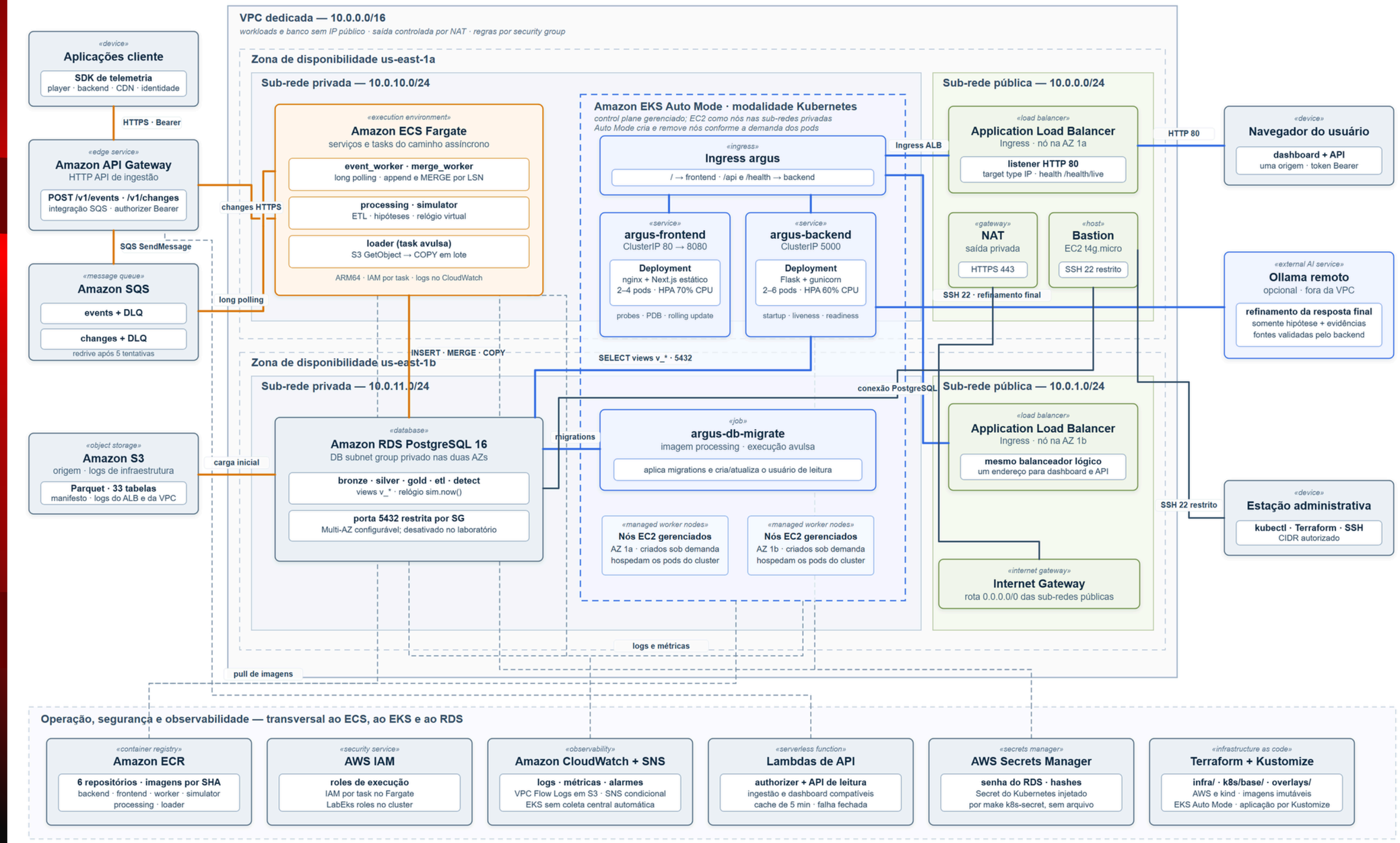
Publicar uma versão nova significava substituir as máquinas inteiras, sem transição suave.

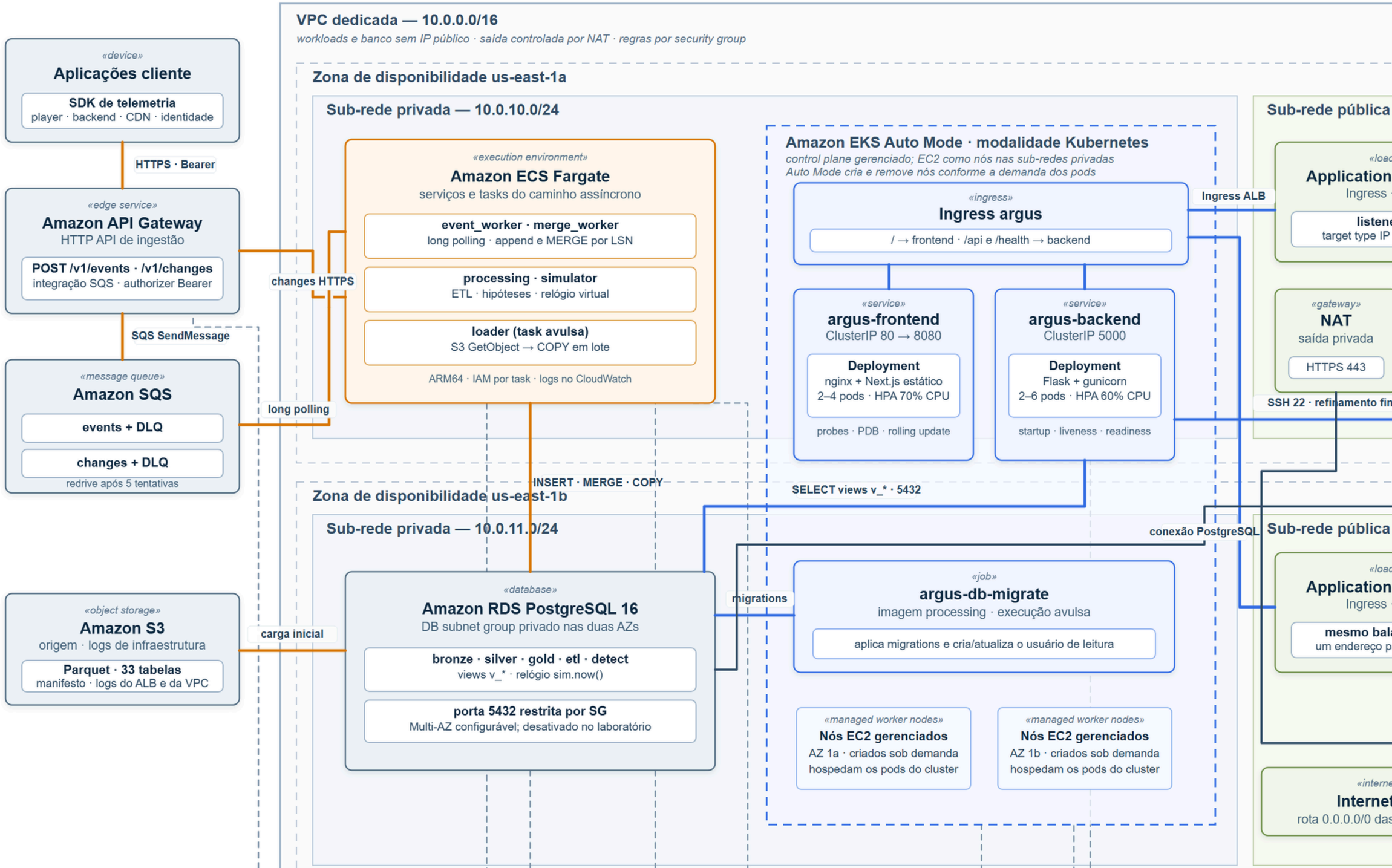
### DASHBOARD E API DE LUGARES DISTINTOS

O painel vinha de um lugar e os dados de outro, e o painel carregava o endereço dos dados gravado dentro dele.

# Diagrama de implantação do Argus — arquitetura final

AWS us-east-1 · VPC em duas AZs · ECS Fargate no caminho assíncrono · EKS Auto Mode com nós EC2 no caminho síncrono




VPC dedicada — 10.0.0.0/16
workloads e banco sem IP público · saída controlada por NAT · regras por security group
«device»
Aplicações cliente
SDK de telemetria
player · backend · CDN · identidade
HTTPS · Bearer
«edge service»
Amazon API Gateway
HTTP API de ingestão
POST /v1/events · /v1/changes
integração SQS · authorizer Bearer
SQS SendMessage
«message queue»
Amazon SQS
events + DLQ
changes + DLQ
redrive após 5 tentativas
«object storage»
Amazon S3
origem · logs de infraestrutura
Parquet · 33 tabelas
manifesto · logs do ALB e da VPC
carga inicial
Zona de disponibilidade us-east-1a
Sub-rede privada — 10.0.10.0/24
changes HTTPS
long polling
«execution environment»
Amazon ECS Fargate
serviços e tasks do caminho assíncrono
event_worker · merge_worker
long polling · append e MERGE por LSN
processing · simulator
ETL · hipóteses · relógio virtual
loader (task avulsa)
S3 GetObject → COPY em lote
ARM64 · IAM por task · logs no CloudWatch
Amazon EKS Auto Mode · modalidade Kubernetes
control plane gerenciado; EC2 como nós nas sub-redes privadas
Auto Mode cria e remove nós conforme a demanda dos pods
«ingress»
Ingress argus
/ → frontend · /api e /health → backend
Ingress ALB
«service»
argus-frontend
ClusterIP 80 → 8080
Deployment
nginx + Next.js estático
2–4 pods · HPA 70% CPU
probes · PDB · rolling update
«service»
argus-backend
ClusterIP 5000
Deployment
Flask + gunicorn
2–6 pods · HPA 60% CPU
startup · liveness · readiness
Sub-rede pública
«load
Application
Ingress
listene
target type IP
«gateway»
NAT
saída privada
HTTPS 443
SSH 22 · refinamento fin
INSERT · MERGE · COPY
SELECT views v_* · 5432
Zona de disponibilidade us-east-1b
Sub-rede privada — 10.0.11.0/24
«database»
Amazon RDS PostgreSQL 16
DB subnet group privado nas duas AZs
bronze · silver · gold · etl · detect
views v_* · relógio sim.now()
porta 5432 restrita por SG
Multi-AZ configurável; desativado no laboratório
migrations
conexão PostgreSQL
«job»
argus-db-migrate
imagem processing · execução avulsa
aplica migrations e cria/atualiza o usuário de leitura
«managed worker nodes»
Nós EC2 gerenciados
AZ 1a · criados sob demanda
hospedam os pods do cluster
«managed worker nodes»
Nós EC2 gerenciados
AZ 1b · criados sob demanda
hospedam os pods do cluster
Sub-rede pública
«load
Application
Ingress
mesmo bala
um endereço p
«interne
Internet
rota 0.0.0.0/0 das

AWS us-east-1 · VPC em duas AZs · ECS Fargate no caminho assíncrono · EKS Auto Mode com nós EC2 no caminho síncrono

nó/serviço
agrupamento lógico

## VPC dedicada — 10.0.0.0/16

*workloads e banco sem IP público · saída controlada por NAT · regras por security group*

### Zona de disponibilidade us-east-1a

#### Sub-rede privada — 10.0.10.0/24

*«execution environment»*

**Amazon ECS Fargate**

serviços e tasks do caminho assíncrono

- **event_worker · merge_worker**
  long polling · append e MERGE por LSN
- **processing · simulator**
  ETL · hipóteses · relógio virtual
- **loader (task avulsa)**
  S3 GetObject → COPY em lote

ARM64 · IAM por task · logs no CloudWatch

changes HTTPS

long polling

INSERT · MERGE · COPY

#### Amazon EKS Auto Mode · modalidade Kubernetes

*control plane gerenciado; EC2 como nós nas sub-redes privadas*
*Auto Mode cria e remove nós conforme a demanda dos pods*

*«ingress»*

**Ingress argus**

/ → frontend · /api e /health → backend

Ingress ALB

*«service»*

**argus-frontend**

ClusterIP 80 → 8080

**Deployment**
nginx + Next.js estático
2–4 pods · HPA 70% CPU

probes · PDB · rolling update

*«service»*

**argus-backend**

ClusterIP 5000

**Deployment**
Flask + gunicorn
2–6 pods · HPA 60% CPU

startup · liveness · readiness

SELECT views v_* · 5432

*«managed worker nodes»*

**Nós EC2 gerenciados**

AZ 1a · criados sob demanda
hospedam os pods do cluster

*«managed worker nodes»*

**Nós EC2 gerenciados**

AZ 1b · criados sob demanda
hospedam os pods do cluster

#### Sub-rede pública — 10.0.0.0/24

*«load balancer»*

**Application Load Balancer**

Ingress · nó na AZ 1a

**listener HTTP 80**
target type IP · health /health/live

HTTP 80

*«gateway»*

**NAT**

saída privada

HTTPS 443

*«host»*

**Bastion**

EC2 t4g.micro

SSH 22 restrito

SSH 22 · refinamento final

### Zona de disponibilidade us-east-1b

#### Sub-rede privada — 10.0.11.0/24

*«database»*

**Amazon RDS PostgreSQL 16**

DB subnet group privado nas duas AZs

- **bronze · silver · gold · etl · detect**
  views v_* · relógio sim.now()
- **porta 5432 restrita por SG**
  Multi-AZ configurável; desativado no laboratório

carga inicial

migrations

*«job»*

**argus-db-migrate**

imagem processing · execução avulsa

aplica migrations e cria/atualiza o usuário de leitura

conexão PostgreSQL

#### Sub-rede pública — 10.0.1.0/24

*«load balancer»*

**Application Load Balancer**

Ingress · nó na AZ 1b

**mesmo balanceador lógico**
um endereço para dashboard e API

*«internet gateway»*

**Internet Gateway**

rota 0.0.0.0/0 das sub-redes públicas

SSH 22 restrito

logs e métricas

pull de imagens

*«device»*

**Navegador do usuário**

**dashboard + API**
uma origem · token Bearer

*«external AI service»*

**Ollama remoto**

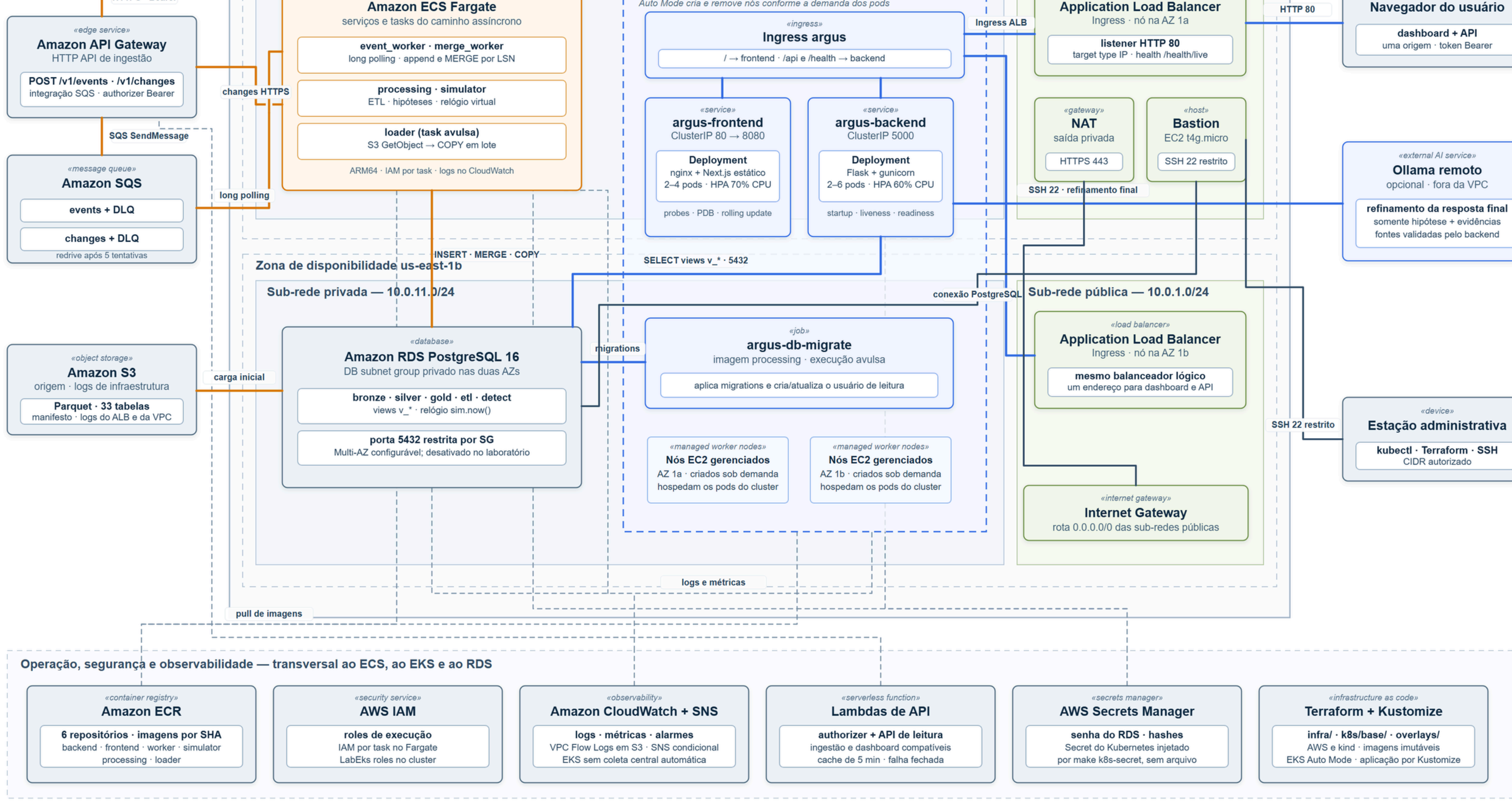
opcional · fora da VPC

**refinamento da resposta final**
somente hipótese + evidências
fontes validadas pelo backend

*«device»*

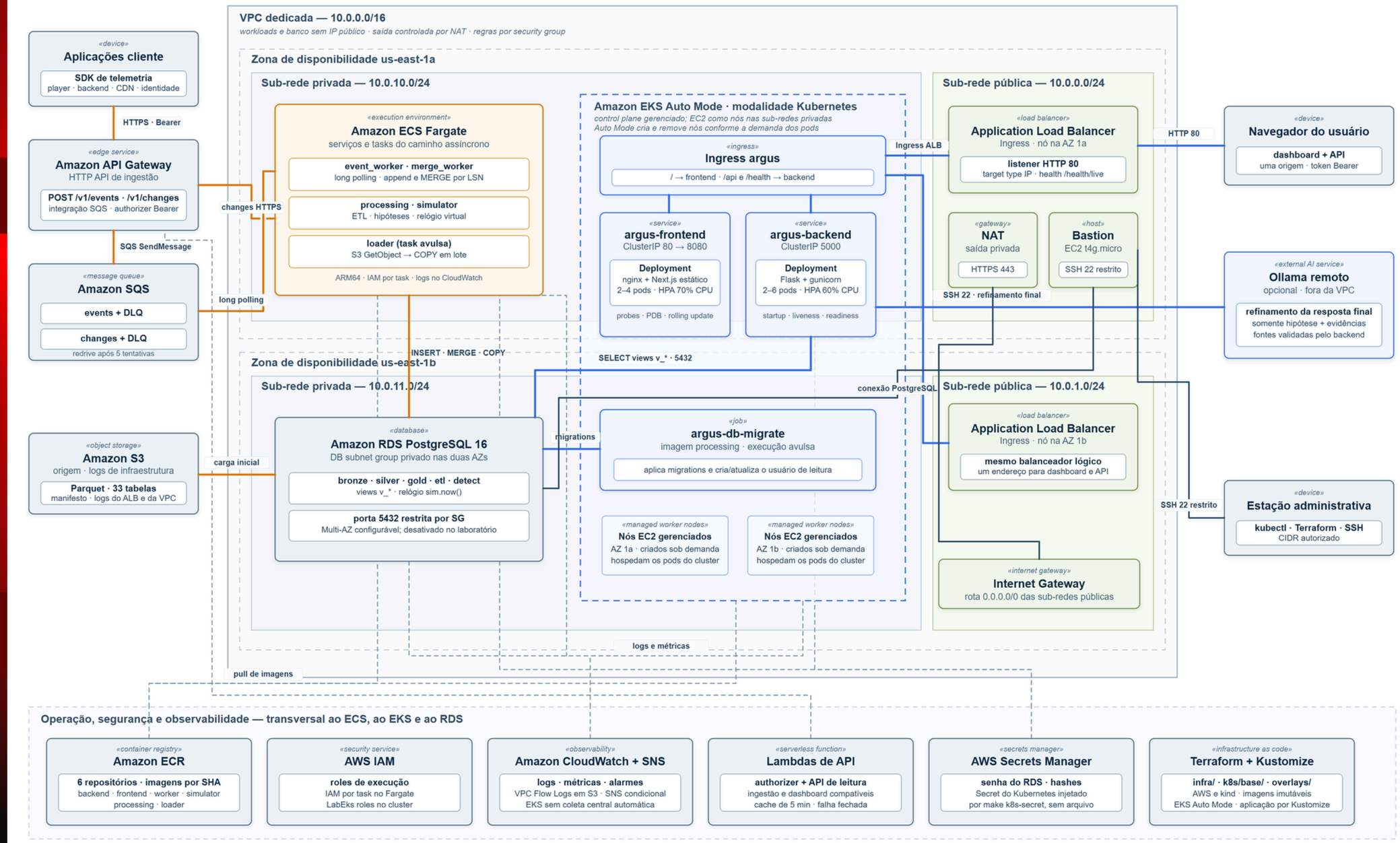
**Estação administrativa**

**kubectl · Terraform · SSH**
CIDR autorizado


«edge service»
Amazon API Gateway
HTTP API de ingestão
POST /v1/events · /v1/changes
integração SQS · authorizer Bearer
changes HTTPS
SQS SendMessage
«message queue»
Amazon SQS
events + DLQ
changes + DLQ
redrive após 5 tentativas
long polling
«object storage»
Amazon S3
origem · logs de infraestrutura
Parquet · 33 tabelas
manifesto · logs do ALB e da VPC
carga inicial
Amazon ECS Fargate
serviços e tasks do caminho assíncrono
event_worker · merge_worker
long polling · append e MERGE por LSN
processing · simulator
ETL · hipóteses · relógio virtual
loader (task avulsa)
S3 GetObject → COPY em lote
ARM64 · IAM por task · logs no CloudWatch
INSERT · MERGE · COPY
Zona de disponibilidade us-east-1b
Sub-rede privada — 10.0.11.0/24
«database»
Amazon RDS PostgreSQL 16
DB subnet group privado nas duas AZs
bronze · silver · gold · etl · detect
views v_* · relógio sim.now()
porta 5432 restrita por SG
Multi-AZ configurável; desativado no laboratório
migrations
Auto Mode cria e remove nós conforme a demanda dos pods
«ingress»
Ingress argus
/ → frontend · /api e /health → backend
«service»
argus-frontend
ClusterIP 80 → 8080
Deployment
nginx + Next.js estático
2–4 pods · HPA 70% CPU
probes · PDB · rolling update
«service»
argus-backend
ClusterIP 5000
Deployment
Flask + gunicorn
2–6 pods · HPA 60% CPU
startup · liveness · readiness
SELECT views v_* · 5432
«job»
argus-db-migrate
imagem processing · execução avulsa
aplica migrations e cria/atualiza o usuário de leitura
«managed worker nodes»
Nós EC2 gerenciados
AZ 1a · criados sob demanda
hospedam os pods do cluster
«managed worker nodes»
Nós EC2 gerenciados
AZ 1b · criados sob demanda
hospedam os pods do cluster
Ingress ALB
conexão PostgreSQL
Sub-rede pública — 10.0.1.0/24
«load balancer»
Application Load Balancer
Ingress · nó na AZ 1a
listener HTTP 80
target type IP · health /health/live
HTTP 80
«gateway»
NAT
saída privada
HTTPS 443
«host»
Bastion
EC2 t4g.micro
SSH 22 restrito
SSH 22 · refinamento final
«load balancer»
Application Load Balancer
Ingress · nó na AZ 1b
mesmo balanceador lógico
um endereço para dashboard e API
«internet gateway»
Internet Gateway
rota 0.0.0.0/0 das sub-redes públicas
Navegador do usuário
dashboard + API
uma origem · token Bearer
«external AI service»
Ollama remoto
opcional · fora da VPC
refinamento da resposta final
somente hipótese + evidências
fontes validadas pelo backend
SSH 22 restrito
«device»
Estação administrativa
kubectl · Terraform · SSH
CIDR autorizado
logs e métricas
pull de imagens
Operação, segurança e observabilidade — transversal ao ECS, ao EKS e ao RDS
«container registry»
Amazon ECR
6 repositórios · imagens por SHA
backend · frontend · worker · simulator
processing · loader
«security service»
AWS IAM
roles de execução
IAM por task no Fargate
LabEks roles no cluster
«observability»
Amazon CloudWatch + SNS
logs · métricas · alarmes
VPC Flow Logs em S3 · SNS condicional
EKS sem coleta central automática
«serverless function»
Lambdas de API
authorizer + API de leitura
ingestão e dashboard compatíveis
cache de 5 min · falha fechada
«secrets manager»
AWS Secrets Manager
senha do RDS · hashes
Secret do Kubernetes injetado
por make k8s-secret, sem arquivo
«infrastructure as code»
Terraform + Kustomize
infra/ · k8s/base/ · overlays/
AWS e kind · imagens imutáveis
EKS Auto Mode · aplicação por Kustomize

# Diagrama de implantação do Argus — arquitetura final

AWS us-east-1 · VPC em duas AZs · ECS Fargate no caminho assíncrono · EKS Auto Mode com nós EC2 no caminho síncrono



# A ARQUITETURA FINAL

## 01
### PORTA ÚNICA

**Um endereço só atende o painel e os dados. Sem conversa entre dois endereços diferentes.**

## 02
### AUTO SCALING

As duas partes rodam em cópias independentes, de 2 a 6, que sobem e descem conforme a procura.

## 03
### PREPARAÇÃO DO BANCO

Uma tarefa curta prepara o banco antes de o serviço começar a atender, e termina sozinha.

## 04
### ENTRADA DE DADOS INTOCADA

Fila e processamento continuam onde estavam.

## 05
### UM BANCO SÓ

Tudo lê e escreve no mesmo banco. Uma fonte, sem cópias paralelas.

# A ARQUITETURA FINAL

## 01
### PORTA ÚNICA

Um endereço só atende o painel e os dados. Sem conversa entre dois endereços diferentes.

## 02
### AUTO SCALING

**As duas partes rodam em cópias independentes, de 2 a 6, que sobem e descem conforme a procura.**

## 03
### PREPARAÇÃO DO BANCO

Uma tarefa curta prepara o banco antes de o serviço começar a atender, e termina sozinha.

## 04
### ENTRADA DE DADOS INTOCADA

Fila e processamento continuam onde estavam.

## 05
### UM BANCO SÓ

Tudo lê e escreve no mesmo banco. Uma fonte, sem cópias paralelas.

# A ARQUITETURA FINAL

## 01
### PORTA ÚNICA

Um endereço só atende o painel e os dados. Sem conversa entre dois endereços diferentes.

## 02
### AUTO SCALING

As duas partes rodam em cópias independentes, de 2 a 6, que sobem e descem conforme a procura.

## 03
### PREPARAÇÃO DO BANCO

**Uma tarefa curta prepara o banco antes de o serviço começar a atender, e termina sozinha.**

## 04
### ENTRADA DE DADOS INTOCADA

Fila e processamento continuam onde estavam.

## 05
### UM BANCO SÓ

Tudo lê e escreve no mesmo banco. Uma fonte, sem cópias paralelas.

# A ARQUITETURA FINAL

## 01
### PORTA ÚNICA

Um endereço só atende o painel e os dados. Sem conversa entre dois endereços diferentes.

## 02
### AUTO SCALING

As duas partes rodam em cópias independentes, de 2 a 6, que sobem e descem conforme a procura.

## 03
### PREPARAÇÃO DO BANCO

Uma tarefa curta prepara o banco antes de o serviço começar a atender, e termina sozinha.

## 04
### ENTRADA DE DADOS INTOCADA

**Fila e processamento continuam onde estavam.**

## 05
### UM BANCO SÓ

Tudo lê e escreve no mesmo banco. Uma fonte, sem cópias paralelas.

# A ARQUITETURA FINAL

## 01
### PORTA ÚNICA

Um endereço só atende o painel e os dados. Sem conversa entre dois endereços diferentes.

## 02
### AUTO SCALING

As duas partes rodam em cópias independentes, de 2 a 6, que sobem e descem conforme a procura.

## 03
### PREPARAÇÃO DO BANCO

Uma tarefa curta prepara o banco antes de o serviço começar a atender, e termina sozinha.

## 04
### ENTRADA DE DADOS INTOCADA

Fila e processamento continuam onde estavam.

## 05
### UM BANCO SÓ

**Tudo lê e escreve no mesmo banco. Uma fonte, sem cópias paralelas.**

# RESULTADOS JÁ OBTIDOS

## 01

### Resultado obtido

Um lote controlado de 52 mudanças **percorreu ingestão, Bronze, Silver e Gold.** O detector **encontrou** duas anomalias no incidente e **nenhuma** no controle, **gerando uma hipótese** de falha interna de backend.

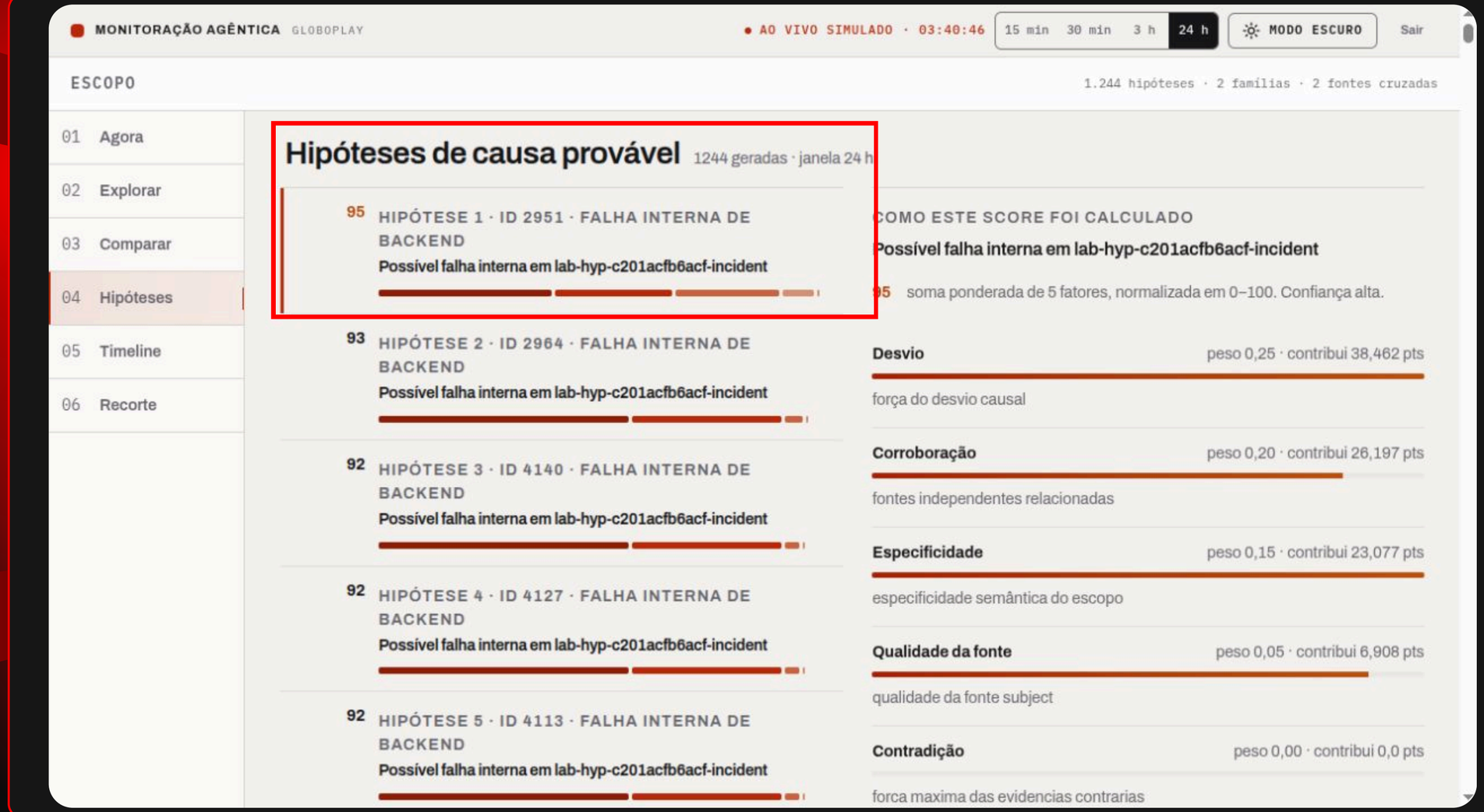
# RESULTADOS JÁ OBTIDOS

## 02

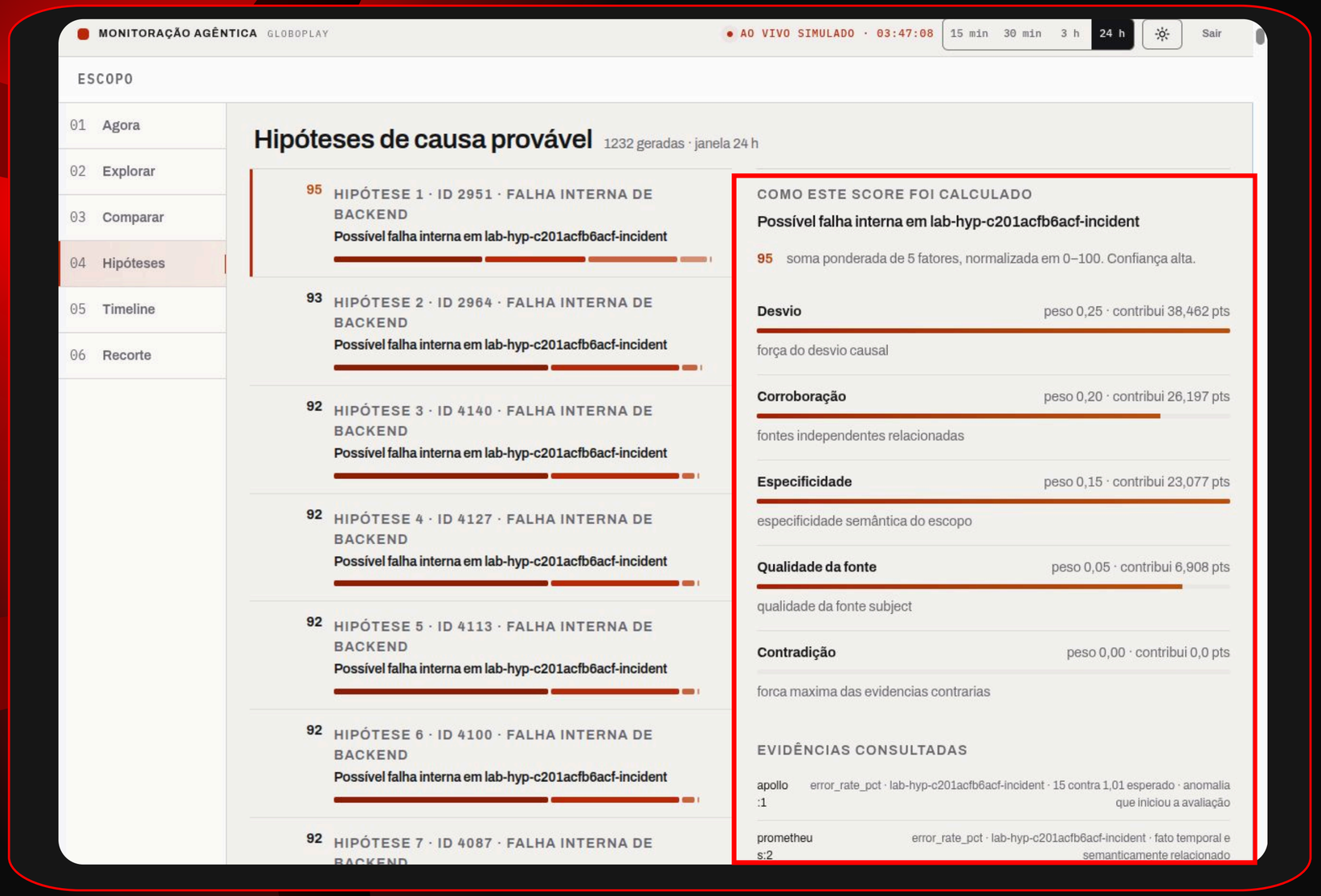
### Saída apresentada

A hipótese 2951 **reuniu evidências** de Apollo e Prometheus. O teste **registrou score** 94,64/100 e cobertura 81,25%; o painel **exibe o score** arredondado para 95, seus **fatores e as evidências.**

# HIPÓTESE 2951 NA LISTA DO FRONTEND



# FATORES E EVIDÊNCIAS DA HIPÓTESE 2951



# RESULTADOS JÁ OBTIDOS

## 03

### Aonde queremos chegar

O fluxo ponta a ponta foi **demonstrado** neste cenário controlado. A próxima etapa é validá-lo com incidentes **variados e rotulados**, medindo **precisão, falsos positivos e tempo até a hipótese.**

# CONTEINERIZAÇÃO DO PROJETO

# O QUE É UM CONTÊINER?

A aplicação empacotada com tudo de que precisa para rodar. Roda igual no computador de quem desenvolve e no servidor.

# O QUE É KUBERNETES?

O gerente dos pacotes. Você diz quantas cópias quer no ar e ele cuida do resto: sobe, vigia, substitui o que quebrou e ajusta a quantidade.

# O QUE MUDOU, NA PRÁTICA

~~5 min~~ →

**4 s**

**TEMPO PARA REPOR UMA CÓPIA QUE CAIU**

~~interrupção~~ →

**0 falhas**

**ATUALIZAÇÃO COM O SISTEMA EM USO**

547 requisições durante uma publicação, nenhuma perdida

~~2 endereços~~ →

**1 endereço**

**ENDEREÇO DE ENTRADA DO USUÁRIO**

O sistema também passou a distinguir “o serviço está vivo” de “o serviço está pronto para atender”. É essa diferença que evita derrubar tudo quando só o banco está indisponível.

# TESTES DA SOLUÇÃO

# TESTES UNITÁRIOS

## O QUE É

Pegamos uma peça pequena do código por vez, damos uma entrada conhecida e conferimos a saída.

## POR QUE IMPORTA

É o teste mais barato de rodar e o mais rápido de consertar. Quando quebra, já sabemos qual linha olhar.

# TESTES UNITÁRIOS

| 133 | 133 | 0 | 4 |
| --- | --- | --- | --- |
| TESTES EXECUTADOS | APROVADOS | REPROVADOS | PARTES DO SISTEMA COBERTAS |

## EXEMPLOS DO QUE FOI VERIFICADO

### A FRONTEIRA EXATA

Um envio de 4 MB é aceito; o mesmo envio com um byte a mais é recusado. Testamos os dois lados da linha.

### ENTRADA ESTRANHA

Uma data em formato desconhecido é recusada na entrada, em vez de virar lixo dentro do banco.

### DADOS DO OPERADOR

O identificador de quem usa a plataforma é trocado por um código irreversível antes de ser guardado.

# TESTES DE INTEGRAÇÃO E IMPLANTAÇÃO

## O QUE É

Subimos o sistema do jeito que ele vai para o ar, provocamos falhas de propósito e conferimos se ele se recupera sozinho.

## O QUE PROCURAMOS

Publicação de versão nova sem derrubar quem está usando, e recuperação automática quando uma parte cai.

# TESTES DE INTEGRAÇÃO E IMPLANTAÇÃO

**12 / 12**

**VERIFICAÇÕES DE PONTA A PONTA**

O painel inteiro navegado por um robô, com o sistema já no formato novo. Nenhuma falhou.

**547**

**REQUISIÇÕES DURANTE UMA PUBLICAÇÃO E SEM FALHA**

Trocamos a versão com o sistema em uso. Ninguém recebeu erro.

**4 s**

**PARA REPOR UMA CÓPIA DERRUBADA À FORÇA**

Antes, na arquitetura anterior, eram cerca de 5 minutos.

**29**

**ETAPAS DE PREPARAÇÃO DO BANCO**

Aplicadas por uma tarefa que roda uma vez, confere e termina sozinha.

# TESTES DE CARGA

**5**
**USUÁRIOS**
1.040
requisições

**25**
**USUÁRIOS**
5.208
requisições

**100**
**USUÁRIOS**
19.253
requisições

## CRITÉRIOS DE APROVAÇÃO

- Menos de 1% de erro
- Resposta abaixo de 3 segundos na maioria das vezes
- Tela completa em menos de 5 segundos
- Mais de 99% das telas abrindo inteiras

# TESTES DE CARGA

**0**

ERROS EM 25.501 REQUISIÇÕES

**0,8s**

PIOR TEMPO DE RESPOSTA TÍPICO, NO DEGRAU MAIS PESADO

**1/6**

DO LIMITE QUE O REQUISITO PERMITE

## NOSSO LIMITE

Repetimos o ensaio subindo a carga até quebrar. Com 100 usuários o sistema passa; com 200, começa a atrasar. O limite está nessa faixa. E é o limite de duas cópias, porque o ajuste automático de quantidade não chegou a entrar em ação nessa rodada.

# NOSSO TIME

vale a pena ver de novo



Christian Gandra



Diego Figueiredo



Gabriel Bartmanovicz



Lívia Tavares



Lorenzo Aggio



Luiz Oliveira



Nicole Riedla



Vinicius Cadena

globoplay

# OBRIGADO!

vale a pena ver de novo